PROJECT REPORT

Global Human Development Index (HDI) Intelligence Engine

A Hybrid Deep Learning Approach for Predicting and Classifying

National Human Development Index Scores

Prepared by (Group Members):

1. Nimmakayala Venkata Satheesh Kumar (Team Lead)
2. Cherukuri Sailaja Kiranmai (Group Member)

Platform: Kaggle Notebook (Dual Tesla T4 GPU Environment)

Date: July 2026

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Project Overview

The Global Human Development Index (HDI) Intelligence Engine is a hybrid deep learning system designed to predict a country's Human Development Index score and classify it into a development tier — Low, Medium, High, or Very High — using key socio-economic indicators such as life expectancy, expected years of schooling, and gross national income (GNI) per capita. The project was developed and iteratively refined inside a Kaggle notebook environment leveraging GPU-accelerated data processing (cuDF/RAPIDS) for preprocessing and PyTorch for model training.

The system combines a dual-branch neural network architecture with a lightweight Flask-based web application, allowing end users to interactively enter development indicators and instantly receive a predicted HDI score and tier classification through a browser-accessible dashboard, publicly exposed using LocalTunnel during the notebook session.

1.2 Purpose

The purpose of this project is to demonstrate an end-to-end applied machine learning pipeline — from raw dataset ingestion and GPU-accelerated cleaning, through hybrid neural network design and training, to deployment as an interactive web tool — that can assist researchers, policy analysts, and students in understanding and estimating a nation's human development standing from a small set of measurable indicators. It also serves as a practical demonstration of combining regression and classification objectives within a single predictive model.

2. IDEATION PHASE

2.1 Problem Statement

Human Development Index computation officially requires combining normalized indices of health, education, and income dimensions through a fixed geometric-mean formula published annually by the UNDP. This makes it difficult for a non-specialist — a student, journalist, or regional planner — to quickly estimate or explore "what-if" scenarios (e.g., "what would our HDI look like if schooling improved by two years?") without manually performing the UNDP calculation across multiple normalized sub-indices. There is a need for a fast, approximate, and interactive tool that can learn the underlying relationship between core indicators and the final HDI score/tier directly from historical data.

2.2 Empathy Map Canvas

The empathy map below captures the perspective of the primary user persona: a policy researcher / development-studies student exploring HDI trends.

Dimension	Insights
Says	"I need a quick way to estimate HDI without redoing the UNDP formula manually."
Thinks	"Which indicators matter most? Can small policy changes shift a country's tier?"
Does	Searches datasets, compares countries in spreadsheets, cross-checks UNDP reports.
Feels	Frustrated by fragmented tools; motivated by clear, visual, instant feedback.
Pain Points	Manual computation, inconsistent data sources, no interactive "what-if" exploration.
Gains Expected	Fast predictions, visual tier classification, ability to test hypothetical indicator values.

2.3 Brainstorming

Candidate ideas generated and evaluated before finalizing the solution approach:

- Train a single-branch regression network on all raw indicators (rejected — treats core and auxiliary indicators with equal weight, diluting signal).
- Use classical regression (Linear/Random Forest) instead of a neural network (considered as baseline; deep hybrid model chosen for extensibility).
- Build a dual-branch hybrid network separating primary UNDP-aligned indicators (life expectancy, schooling, income) from secondary auxiliary indicators — selected as the final approach.
- Expose the trained model only as a static notebook output (rejected — reduces accessibility for non-technical users).
- Deploy a lightweight Flask + HTML dashboard tunneled via LocalTunnel for live, interactive access — selected as the final deployment approach.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

Stage	User Action	Experience
Discovery	User accesses the LocalTunnel-hosted web dashboard link.	Lands on a clean, single-page HDI prediction interface.
Input	Enters life expectancy, schooling, and income values.	Simple numeric form with validation ranges.
Prediction	Submits the form to trigger model inference.	Real-time HDI score and tier badge returned within seconds.
Interpretation	Reviews the predicted tier against benchmark table.	Colour-coded tier badge (red/amber/blue/green) aids quick understanding.
Iteration	Adjusts indicator values to explore what-if scenarios.	Instant recalculation without page reload delay.

3.2 Solution Requirement

Functional and non-functional requirements identified for the system:

- Ingest and clean HDI indicator datasets automatically, handling missing values on the GPU.
- Dynamically detect relevant columns (life expectancy, schooling, income, HDI target) regardless of exact naming variations across dataset versions.
- Train a hybrid neural network capable of both regression (HDI score) and derived classification (development tier).
- Persist trained weights and feature-scaling parameters for reproducible inference.
- Expose the trained model through a web-based interface accessible without local installation.
- Achieve low regression error (MAE/RMSE) and acceptable classification accuracy across all four tiers.

3.3 Data Flow Diagram

Figure 3.1 — Data Flow Diagram (Level 1)

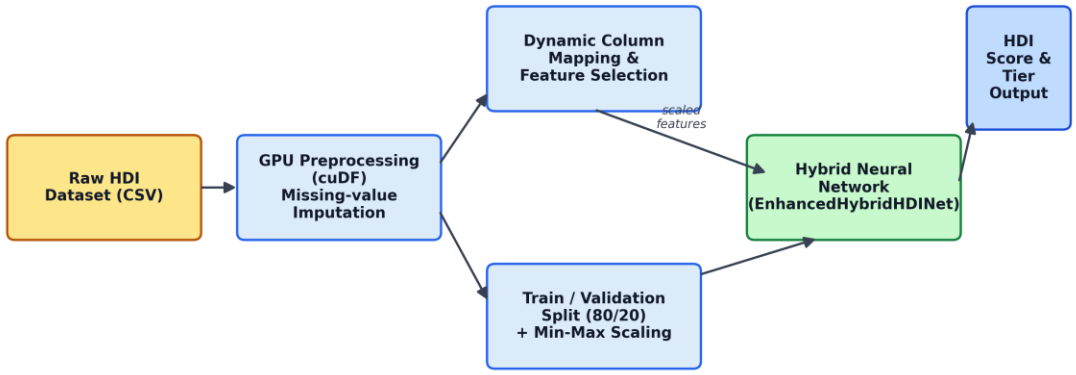

Figure 3.1: Data Flow Diagram showing the flow from raw dataset to predicted HDI output.

3.4 Technology Stack

Layer	Technology Used
Data Processing	cuDF (RAPIDS GPU DataFrame), Pandas, NumPy
Model Development	PyTorch (torch.nn, AdamW optimizer)
Evaluation & Metrics	scikit-learn (classification_report, confusion_matrix, roc_curve, auc)
Visualization	Matplotlib, Seaborn
Backend / API	Flask (Python web framework)

Layer	Technology Used
Frontend	HTML5, CSS3 (Jinja2 templating), Google Fonts (Inter)
Deployment / Tunnelling	LocalTunnel (via npx), Kaggle Notebook GPU runtime (Tesla T4)
Packaging	Python zipfile module for artifact bundling

4. PROJECT DESIGN

4.1 Problem – Solution Fit

The problem of inaccessible, manually-computed HDI estimation is addressed by a solution that fits the target users' workflow closely: a trained model reproduces the relationship between core UNDP indicators and the final HDI score/tier with a Mean Absolute Error of 0.026 and 83% tier classification accuracy on held-out test data, while the web dashboard removes the need for any manual calculation or spreadsheet work, delivering results in real time through a browser.

4.2 Proposed Solution

Parameter	Description
Problem	Manual, formula-heavy HDI estimation with no interactive exploration tool.
Solution	Hybrid neural network trained on historical HDI data, served via a Flask web dashboard.
Novelty	Dual-branch architecture separating core UNDP indicators from auxiliary features; combined regression + tier classification.
Social Impact	Improves accessibility of development-indicator analysis for students, researchers, and policy communicators.
Scalability	Model and scaling parameters are serialized (.pt) and can be redeployed on any Flask-compatible host.

4.3 Solution Architecture

Figure 4.1 — Solution Architecture

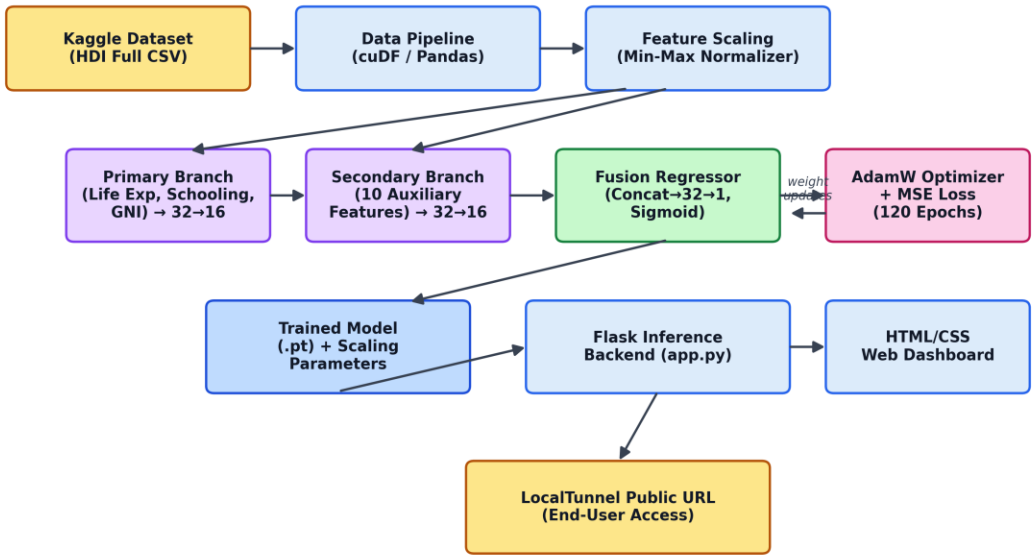

Figure 4.1: End-to-end solution architecture from dataset to deployed web dashboard.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Phase	Activities	Key Outcome
Phase 1 – Data Preparation	Dataset ingestion, GPU-based missing value imputation, dynamic column mapping.	Cleaned, analysis-ready dataset.
Phase 2 – Exploratory Analysis	Scatter plots (Life Expectancy vs HDI, Education vs HDI), correlation heatmap.	Feature relationships identified.
Phase 3 – Model Design	Dual-branch EnhancedHybridHDINet architecture defined (primary + secondary branches).	Model architecture finalized.
Phase 4 – Training	120-epoch training with AdamW optimizer, MSE loss, train/val split, per-epoch logging.	Trained model + convergence curves.
Phase 5 – Evaluation	MAE/RMSE computation, confusion matrix, classification report, ROC-AUC curves.	Verified model performance.
Phase 6 – Deployment	Flask backend + HTML dashboard built and tunnelled via LocalTunnel.	Publicly accessible live demo.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

The trained model was evaluated on a held-out test split of the HDI dataset. Regression and classification metrics obtained are summarized below.

Metric	Value
Mean Absolute Error (MAE)	0.02640
Root Mean Squared Error (RMSE)	0.03113
Overall Classification Accuracy	83%
Macro Average F1-score	0.83
Weighted Average F1-score	0.83

Per-class precision, recall, and F1-score on the test set (n = 141):

Development Tier	Precision	Recall	F1-score	Support
Low	0.84	1.00	0.91	21
Medium	0.89	0.73	0.80	33
High	0.63	0.91	0.74	32
Very High	1.00	0.78	0.88	55

The classification report shows strong precision for the "Very High" tier (1.00) and strong recall for the "Low" tier (1.00), while the "High" tier shows comparatively lower precision (0.63), indicating some over-prediction into this class from adjacent tiers — a reasonable outcome given the continuous, boundary-sensitive nature of tier cut-offs derived from a regression output.

7. RESULTS

7.1 Output Screenshots

Exploratory visualizations generated during data analysis:

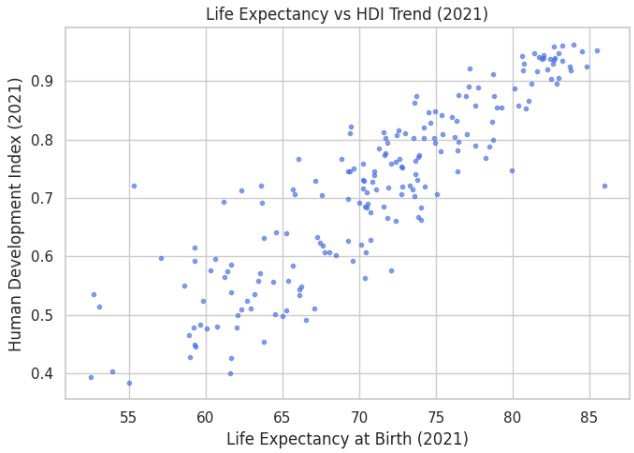

Figure 7.1: Life Expectancy vs HDI trend (2021).

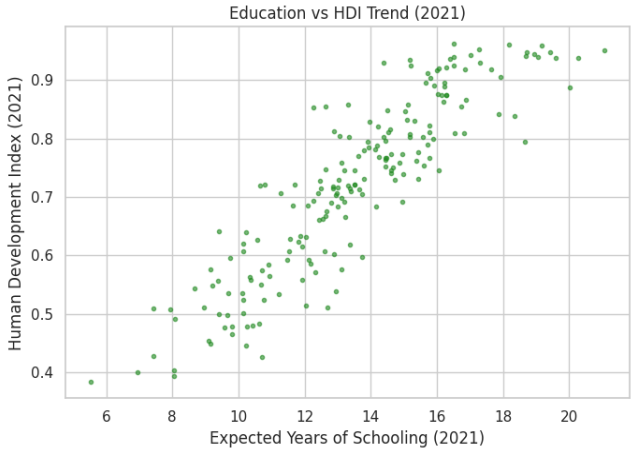

Figure 7.2: Education (Expected Years of Schooling) vs HDI trend (2021).

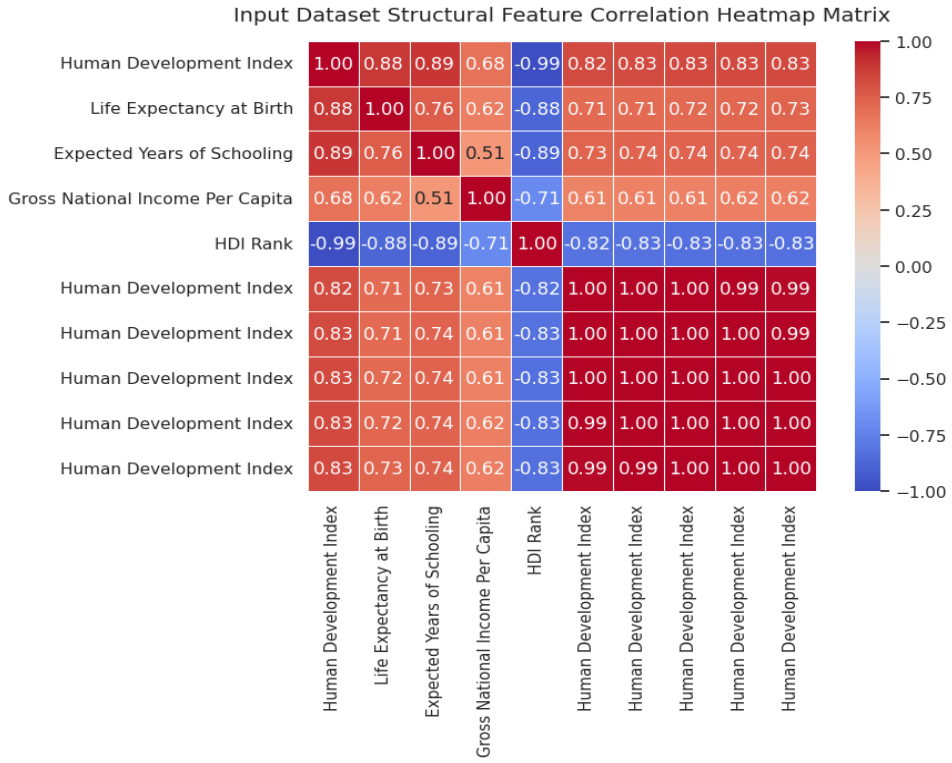

Figure 7.3: Feature correlation heatmap across HDI dataset indicators.

Model training and evaluation visualizations:

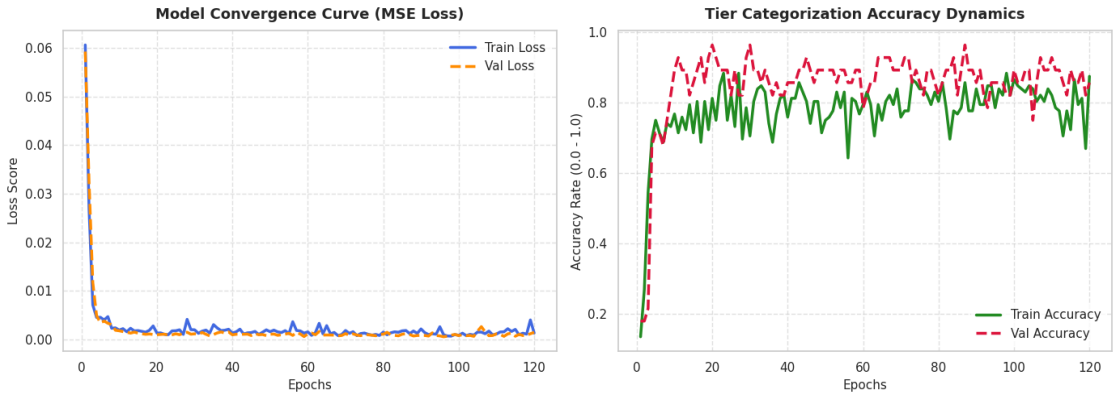

Figure 7.4: Training vs validation loss and accuracy convergence curves over 120 epochs.

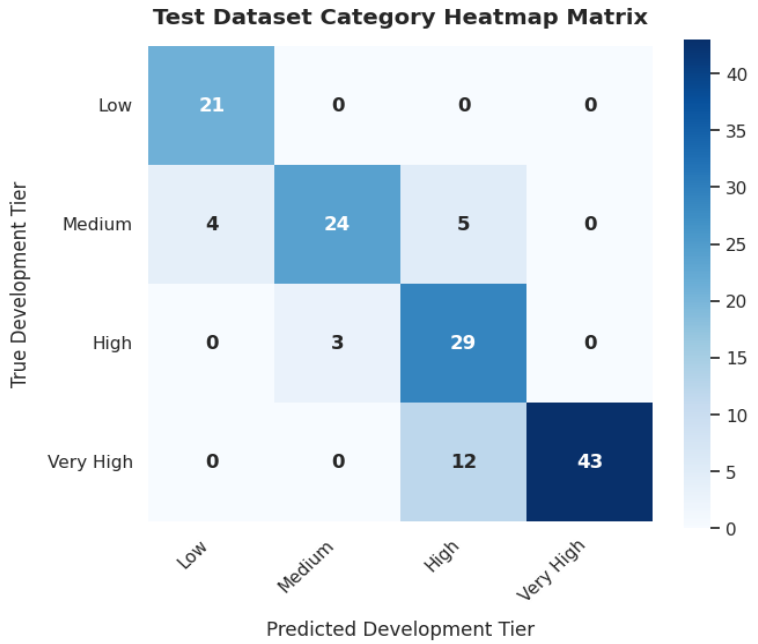

Figure 7.5: Confusion matrix — predicted vs true development tier (training run).

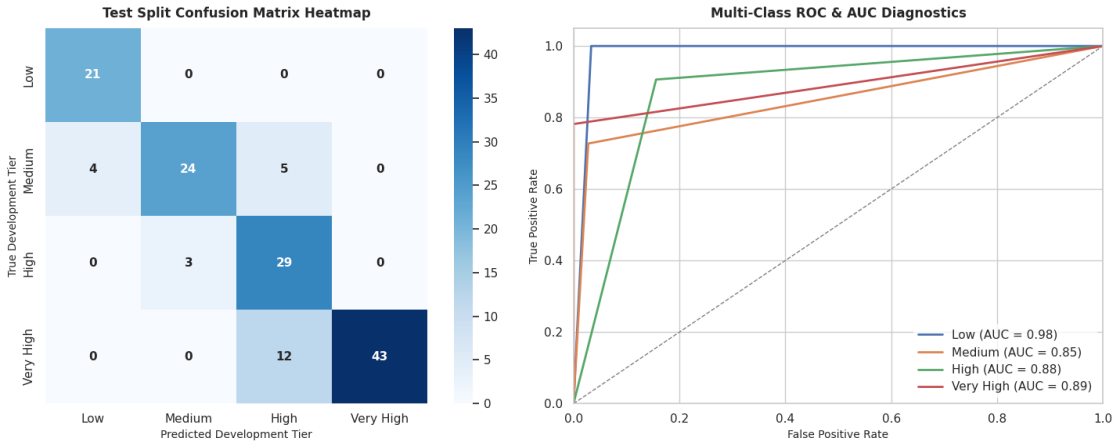

Figure 7.6: Final test-set diagnostics — confusion matrix and multi-class ROC-AUC curves.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Provides near-instant HDI estimation without manual UNDP formula computation.
- Dual-branch architecture separates core indicators from auxiliary ones, improving interpretability of feature contribution.
- Combines regression precision with human-readable tier classification for easier communication of results.
- GPU-accelerated preprocessing (cuDF) enables fast handling of large multi-year datasets.
- Lightweight Flask deployment allows public access without dedicated server infrastructure.

Disadvantages

- Model accuracy is bounded by the quality and completeness of the underlying UNDP dataset; missing-value imputation may introduce bias.
- Tier boundaries are hard thresholds on a continuous prediction, so borderline countries near cut-offs are more prone to misclassification (as seen in the “High” tier's lower precision).
- LocalTunnel-based deployment is temporary and tied to an active Kaggle session; it is not a persistent production deployment.
- The secondary feature branch selects only the first ten available numerical columns programmatically, which may not always capture the most predictive auxiliary indicators.

9. CONCLUSION

The Global HDI Intelligence Engine successfully demonstrates that a compact, dual-branch neural network can approximate the UNDP Human Development Index computation with high fidelity, achieving a Mean Absolute Error of 0.026 and an overall tier classification accuracy of 83% on held-out data. By pairing this model with an interactive Flask-based web dashboard, the project bridges the gap between a static machine learning experiment and an accessible, real-time decision-support tool. The end-to-end pipeline — from GPU-accelerated data cleaning through model training, evaluation, and public deployment — validates a practical and reproducible workflow for applied socio-economic prediction tasks.

10. FUTURE SCOPE

- Replace hard tier thresholds with a learned, soft classification head trained jointly with the regression objective to reduce boundary misclassification.

- Incorporate multi-year historical data with temporal modeling (e.g., LSTM/Transformer) to forecast future HDI trajectories rather than a single-year snapshot.
- Deploy the Flask application on a persistent cloud host (e.g., a managed container service) instead of a session-bound LocalTunnel URL.
- Add explainability components (e.g., SHAP values) to show users which indicators most influenced a given prediction.
- Extend the secondary feature branch with a principled feature-selection step rather than a positional column slice.

11. APPENDIX

Source Code (Key Model Definition)

```
class EnhancedHybridHDINet(nn.Module):     def __init__(self, primary_dim,
secondary_dim):         super().__init__()          self.primary_branch =
nn.Sequential(             nn.Linear(primary_dim, 32), nn.BatchNorm1d(32), nn.ReLU(),
nn.Linear(32, 16), nn.ReLU())         self.secondary_branch = nn.Sequential(
nn.Linear(secondary_dim, 32), nn.BatchNorm1d(32), nn.ReLU(),             nn.Linear(32,
16), nn.ReLU())         self.regressor = nn.Sequential(             nn.Linear(16 + 16,
32), nn.ReLU(), nn.Dropout(0.1),             nn.Linear(32, 1), nn.Sigmoid())      def
forward(self, x_prim, x_sec):         p_out = self.primary_branch(x_prim)
s_out = self.secondary_branch(x_sec)          combined = torch.cat((p_out, s_out),
dim=1)         return self.regressor(combined)
```

Dataset Link

Human Development Index – Full Dataset (Kaggle): https://www.kaggle.com/datasets/iamsouravbanerjee/human-development-index-dataset

GitHub & Project Demo Link

GitHub Repository: [GitHub - sailajakiranmai06/Human-Development-Index-Predictor: "HDI Predictor - A machine learning web application to predict Human Development Index of countries using Python, Flask, and Scikit-learn. Built as part of APSCHE SmartBridge AI/ML Internship." · GitHub]

Live Demo: [Human Development Index Predictor]